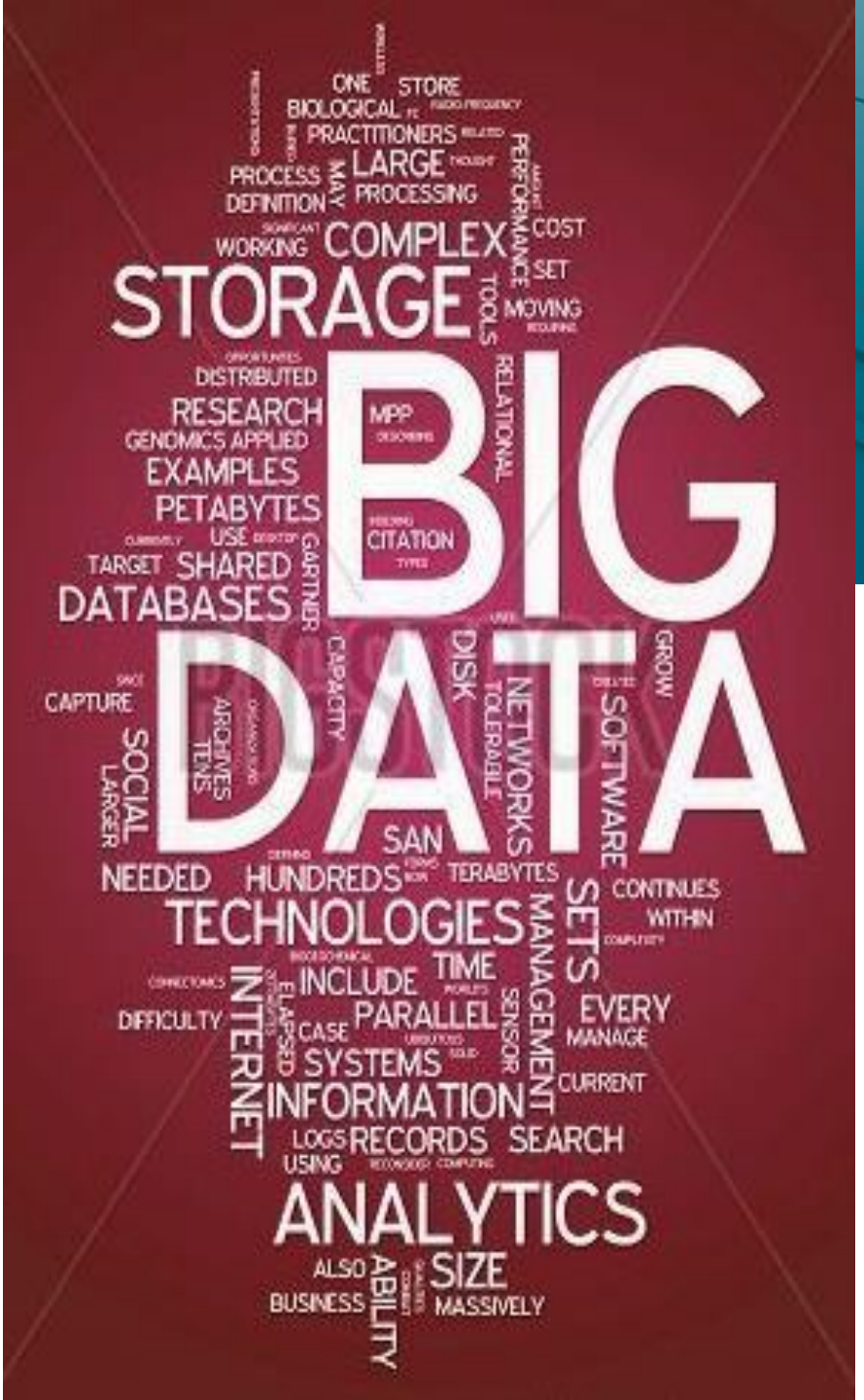


IMSE 785

Big Data Analytics

Mapreduce Steaming

Shing Chang, MH Amini

IMSE, K-State

4.1 Hadoop Streaming

- A Framework for Python and Hadoop Streaming
- Ref. Chap 3. Bengfort and Kim (2016), Data Analytics with Hadoop
- See the Canvas page called “Word Count with Python”

- Wordcount program revisted
- Two key components in MapReduce:
  - A mapper function
  - A reduce function
- We want to run Python on both functions by using the Unix piping concept
  - The main idea is for debugging purposes
  - Stdin and stdout are used for both functions
- Once both functions were tested, all you have to do is to specify the input file and output folder in the HDFS and run it

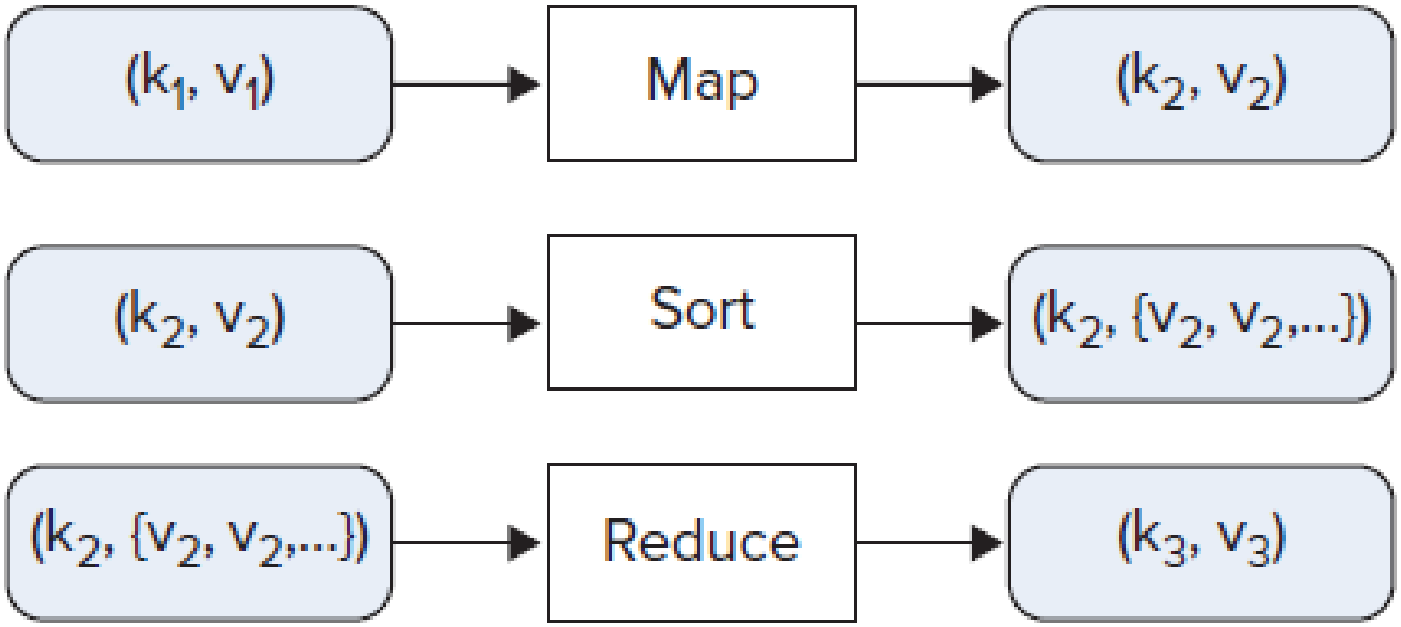
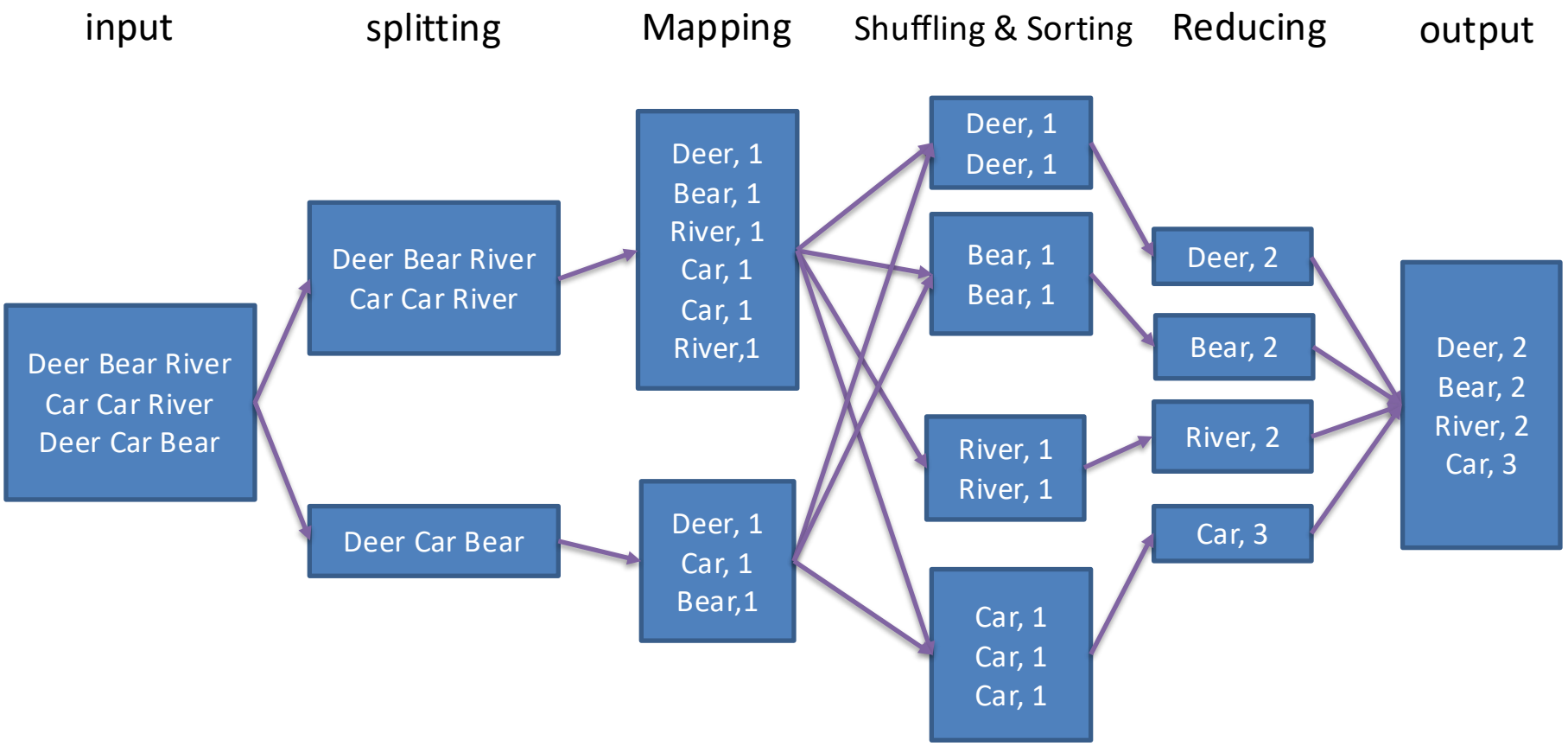


FIGURE 3-1: The functionality of mappers and reducers

Wordcount Example



- The map function in Python (mapper.py):

```
#!/usr/bin/env python
import sys

#--- get all lines from stdin ---
for line in sys.stdin:
    #--- remove leading and trailing whitespace---
    line = line.strip()

    #--- split the line into words ---
    words = line.split()

    #--- output tuples [word, 1] in tab-delimited format---
    for word in words:
        #print '%s\t%s' % (word, "1") #python format; python 3 uses the following format
        print("{}\t{}".format(word, 1))
```

Deer Bear River
Car Car River
Deer Car Bear

- The reduce function in Python (reducer.py):

```
#!/usr/bin/env python
import sys

# maps words to their counts
word2count = {}

# input comes from STDIN
for line in sys.stdin:
    # remove leading and trailing whitespace # parse the input we got from mapper.py
    word, count = line.strip().split('\t', 1)

    # convert count (currently a string) to int
    try:
        count = int(count)
    except ValueError:
        continue

    try:
        word2count[word] = word2count[word]+count
    except:
        word2count[word] = count

# write the tuples to stdout
# Note: they are unsorted
for word in word2count.keys():
    # print '%s\t%s'% ( word, word2count[word] )
    #use the following format for Python 3
    print("{}\t{}".format(word, word2count[word]))
```

Debugging in Unix (not in HDFS yet)

- Make both mapper.py and reducer.py executable
  - **chmod +x mapper.py**
  - **chmod +x reducer.py**
- Execute both codes using standard IO
  - **cat text.txt | python3 mapper.py | python3 reducer.py**

Execute map&reduce in HDFS

- Create a bash file called runmr.sh

```
#!/bin/bash
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.0.jar \
        -input /user/changs/text.txt \
        -output /user/changs/output1 \
        -file mapper.py \
        -file reducer.py \
        -mapper "python3 mapper.py" \
        -reducer "python3 reducer.py"
```

- Run this bash file by
  - **bash runmr.sh**
- **Make sure the input file text.txt is in a HDFS directory (this way a big data file can be analyzed)**
- **To run the same bash again, change the output directory or run hdfs dfs –rm –r output1 first.**

4.2 MapReduce Patterns

- Format/Paradigm: (key, val) but what are the design rules?
  - Learn by Examples
- Ref. Chaps 2, 3 & 5. Bengfort and Kim (2016), Data Analytics with Hadoop
- See the Canvas page called “Word Count with Python” & File>MapReduce folders for all python codes demonstrated

Pattern 1. Wordcount
Python pseudocode

```
#emit is a function that performs Hadoop I/O
#emit is similar to yield in Python function
def map(dockey, line):
        for word in line.split():
                emit(word, 1)
def reduce(word, values):
        count=sum(value for value in values)
        emit(word, count)
```

(Key, Val) design for wordcount

- Are the (key, val) the same for mapper and reducer?
- Discussion of (key, val) design for the mapper. Can there be null for the key?

Pattern 2. Shared Friendship Python pseudocode

```
def map(person, friends):
        for friend in friends.split(“,”):
                pair=sort([person, friend])
                emit(pair, friends)
def reduce(pair, friends):
        shared=set(friends[0]) #friends of 1st name in
                pair
        shared=shared.intersection(friends[1])
        emit(pair, shared)
```

Input file

Key	Value
Allen	Betty, Chris, David
Betty	Allen, Chris, David, Ellen
David	Allen, Betty, Chris, Ellen
Ellen	Betty, Chris, David

	Mapper 1 output
Allen, Betty	Betty, Chris, David
Allen, Chris	Betty, Chris, David
Allen, David	Betty, Chris, David

Input file

Key	Value
Allen	Betty, Chris, David
Betty	Allen, Chris, David, Ellen
David	Allen, Betty, Chris, Ellen
Ellen	Betty, Chris, David

	Mapper 2 output
Allen, Betty	Allen, Chris, David, Ellen
Betty, Chris	Allen, Chris, David, Ellen
?	Allen, Chris, David, Ellen
Betty, Ellen	Allen, Chris, David, Ellen

Input file

Key	Value
Allen	Betty, Chris, David
Betty	Allen, Chris, David, Ellen
David	Allen, Betty, Chris, Ellen
Ellen	Betty, Chris, David

	Mapper 3 output
Allen, David	Allen, Betty, Chris, Ellen
?	Allen, Betty, Chris, Ellen
?	Allen, Betty, Chris, Ellen
?	Allen, Betty, Chris, Ellen

Input file

Key	Value
Allen	Betty, Chris, David
Betty	Allen, Chris, David, Ellen
David	Allen, Betty, Chris, Ellen
Ellen	Betty, Chris, David

	Mapper 4 output
Betty, Ellen	Betty, Chris, David
Chris, Ellen	Betty, Chris, David
Ellen, David	Betty, Chris, David

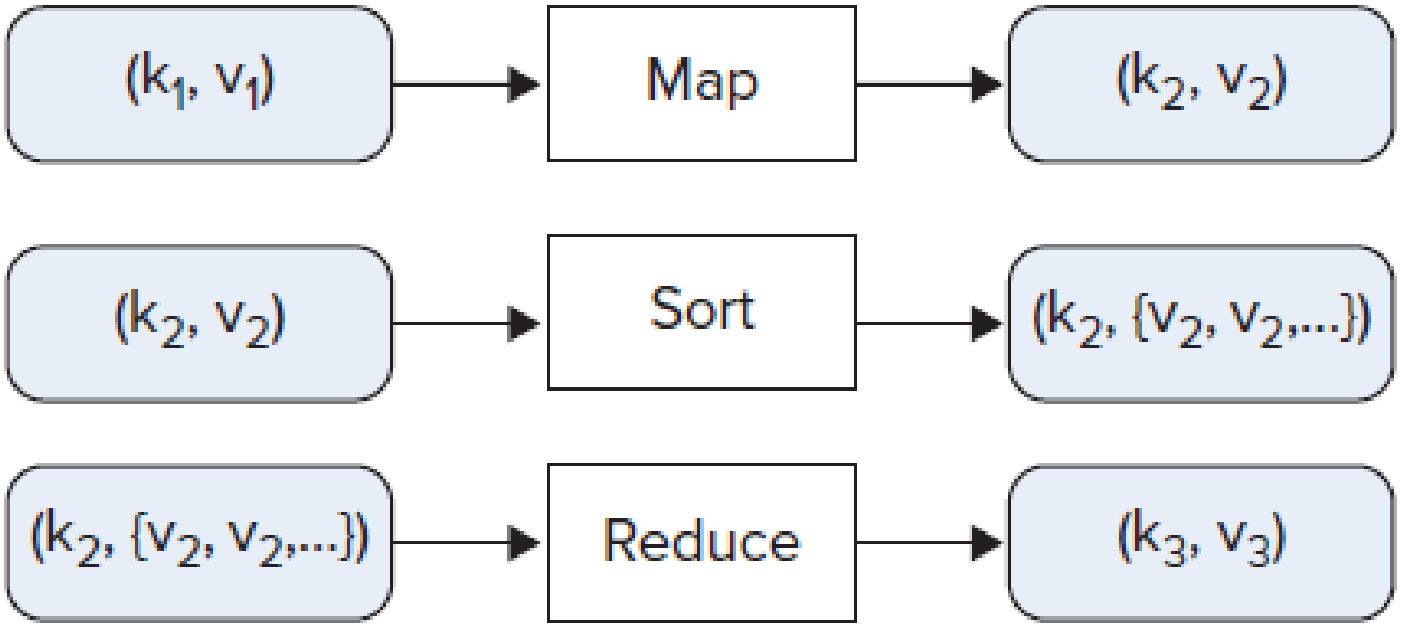


FIGURE 3-1: The functionality of mappers and reducers

After shuffle and sort, reducer input:

Key	Val
Allen, Betty	(A C D E) (B C D)
Allen, Chris	(A B D E) (B C D)
Allen, David	(A B C E) (B C D)
Betty, Chris	?1
Betty, David	(A B C E) (A C D E)
Betty, Ellen	(A C D E)(B C D)
Chris, David	(A B C E) (A B D E)
Chris, Ellen	(A B D E) (B C D)
Ellen, David	(A B C E) (B C D)

After reduction:

Key	Val
Allen, Betty	(A C D E) (B C D) =(Chris, David)
Allen, Chris	(A B D E) (B C D) = ?3
Allen, David	(A B C E) (B C D) = ?4
Betty, Chris	(A B D E) (A C D E)
Betty, David	(A B C E) (A C D E)
Betty, Ellen	(A C D E) (B C D)
Chris, David	(A B C E) (A B D E)
Chris, Ellen	(A B D E) (B C D)
Ellen, David	(A B C E) (B C D)

What question does this output answer?

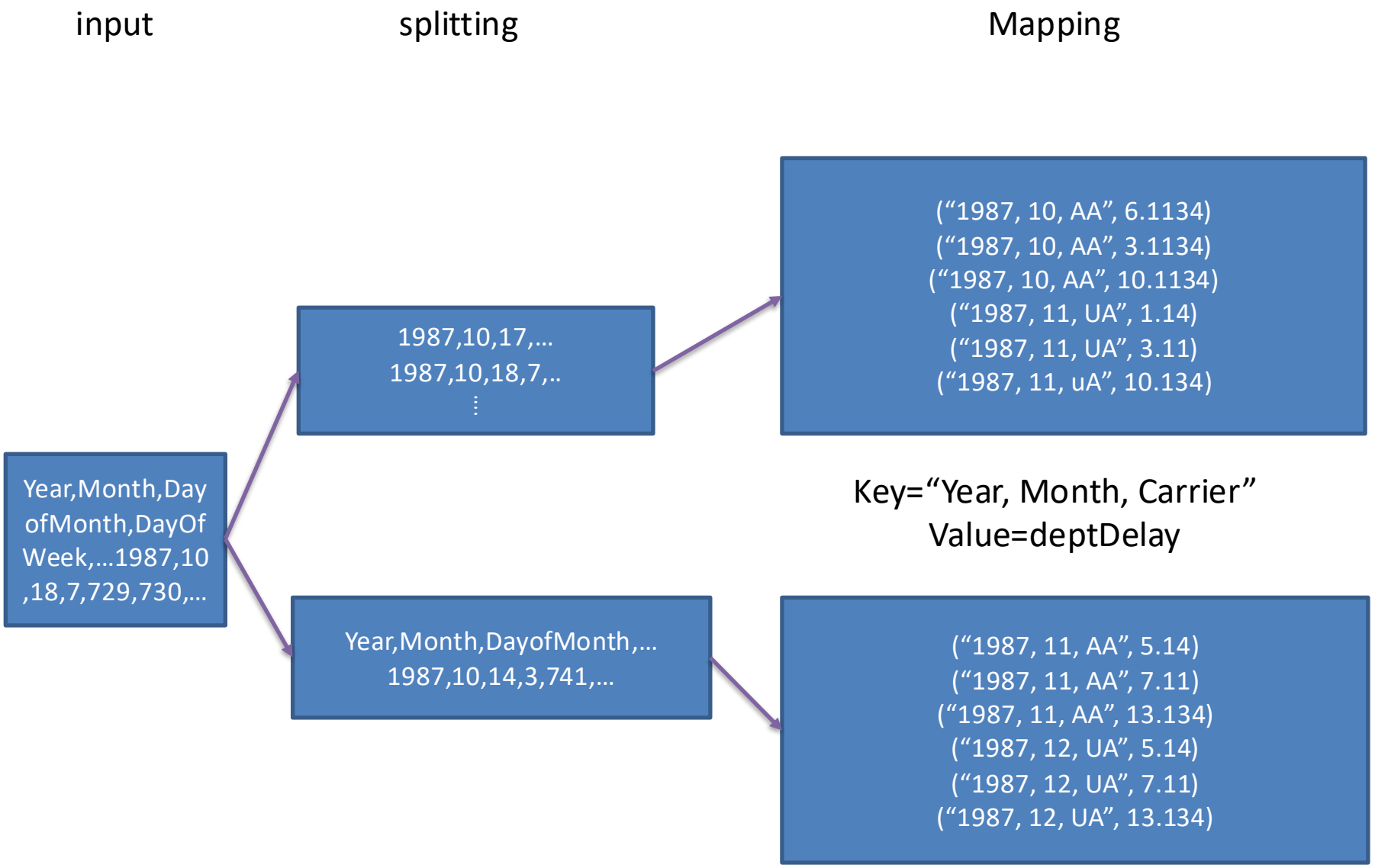
(Key, Val) Design for the friendship pattern

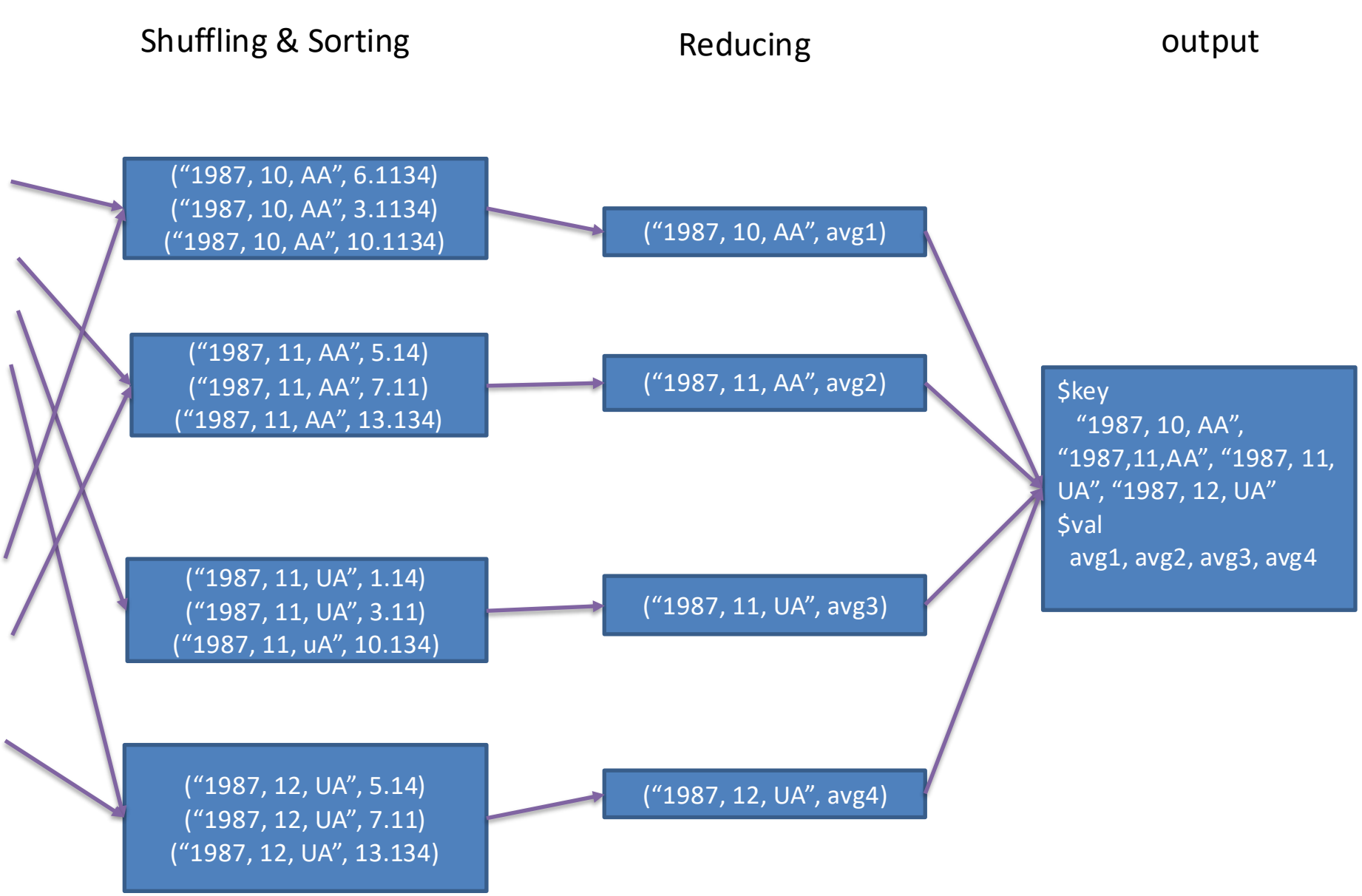
- What are the (key, val) designs for mapper and reducer?

Pattern 3. Fight Delay Analysis

- Goal: to compute average departure delays sorted by departing airports
- Dataset: 1987.csv
- Sample data:
  - Column 0-Year, Column 1-Month
  - Column 16-Origin Airport,
  - Column 15 – Departure Delay

C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
yofMor	DayOfWe	DepTime	CRSDepTi	ArrTime	CRSArrTin	UniqueCa	FlightNun	TailNum	ActualElaj	CRSElapse	AirTime	ArrDelay	DepDelay	Origin	Dest
14	3	741	730	912	849	PS	1451	NA	91	79	NA	23	11	SAN	SFO
15	4	729	730	903	849	PS	1451	NA	94	79	NA	14	-1	SAN	SFO


input
splitting
Mapping
Year,Month,DayofMonth,DayOfWeek,…1987,10,18,7,729,730,…
1987,10,17,…
1987,10,18,7,..
Year,Month,DayofMonth,…
1987,10,14,3,741,…
(“1987, 10, AA”, 6.1134)
(“1987, 10, AA”, 3.1134)
(“1987, 10, AA”, 10.1134)
(“1987, 11, UA”, 1.14)
(“1987, 11, UA”, 3.11)
(“1987, 11, uA”, 10.134)
Key=“Year, Month, Carrier”
Value=deptDelay
(“1987, 11, AA”, 5.14)
(“1987, 11, AA”, 7.11)
(“1987, 11, AA”, 13.134)
(“1987, 12, UA”, 5.14)
(“1987, 12, UA”, 7.11)
(“1987, 12, UA”, 13.134)


Shuffling & Sorting
Reducing
output
("1987, 10, AA", 6.1134)
("1987, 10, AA", 3.1134)
("1987, 10, AA", 10.1134)
("1987, 11, AA", 5.14)
("1987, 11, AA", 7.11)
("1987, 11, AA", 13.134)
("1987, 11, UA", 1.14)
("1987, 11, UA", 3.11)
("1987, 11, uA", 10.134)
("1987, 12, UA", 5.14)
("1987, 12, UA", 7.11)
("1987, 12, UA", 13.134)
("1987, 10, AA", avg1)
("1987, 11, AA", avg2)
("1987, 11, UA", avg3)
("1987, 12, UA", avg4)
$key
"1987, 10, AA",
"1987,11,AA", "1987, 11,
UA", "1987, 12, UA"
$val
avg1, avg2, avg3, avg4

(key, val) design for flight delay pattern

- Goal: to compute average departure delays sorted by departing airports
- Key:
- Val:
- What if you want to sort the results by month? What should the Key and Val be?

- https://github.com/bbengfort/hadoop-fundamentals/blob/master/avgdelay/mapper.py

The map function in Python (mapper3.py):

```
1  #!/usr/bin/env python
2
3  import csv
4  import sys
5
6  SEP = "\t"
7
8  class Mapper(object):
9
10     def __init__(self, stream, sep=SEP):
11         self.stream = stream
12         self.sep    = sep
13
14     def emit(self, key, value):
15         sys.stdout.write("%s%s%s\n" % (key, self.sep, value))
16
17     def map(self):
18         reader = csv.reader(self.stream)
19         for row in reader:
20             self.emit(row[16], row[15])
21
22 if __name__ == '__main__':
23     mapper = Mapper(sys.stdin)
24     mapper.map()
```

- https://github.com/bbengfort/hadoop-fundamentals/blob/master/avgdelay/reducer.py

The reduce function in Python (reducer3.py): 1/2

```
1  #!/usr/bin/env python
2
3  import sys
4
5  from itertools import groupby
6  from operator import itemgetter
7
8  SEP = "\t"
9
10 class Reducer(object):
11
12         def __init__(self, stream, sep=SEP):
13                 self.stream = stream
14                 self.sep    = sep
15
16         def emit(self, key, value):
17                 sys.stdout.write("%s%s%s\n" % (key, self.sep, value))
18
```

- https://github.com/bbengfort/hadoop-fundamentals/blob/master/avgdelay/reducer.py

The reduce function in Python (reducer3.py): 2/2

```
19          def reduce(self):
20                  for current, group in groupby(self, itemgetter(0)):
21                          total = 0
22                          count = 0
23
24                          for item in group:
25                                  total += item[1]
26                                  count += 1
27
28                          self.emit(current, float(total)/float(count))
29
30          def __iter__(self):
31                  for line in self.stream:
32                          try:
33                                  parts = line.split(self.sep)
34                                  yield parts[0], float(parts[1])
35                          except:
36                                  continue
37
38  if __name__ == '__main__':
39          reducer = Reducer(sys.stdin)
40          reducer.reduce()
```

Debugging in Unix (not in HDFS yet)

- Make both mapper.py and reducer.py executable
  - **chmod +x mapper3.py**
  - **chmod +x reducer3.py**
- Execute both codes using standard IO
  - **cat 187.csv | python3 mapper3.py | python3 reducer3.py**
  - **cat 187.csv | python3 mapper3.py | python3 reducer3.py > delay-output.txt**

Your Turn

- Run the following command:
- **cat 187.csv | python3 mapper3.py | sort | python3 reducer3.py > delay-output2.txt**
- **Compare delay-output.txt and delay-output2.txt. What do you find?**

Execute map&reduce in HDFS

- Create a bash file called runmr.sh

```
#!/bin/bash
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.0.jar \
        -input /user/changs/flight/187.csv \
        -output /user/changs/flight/delay \
        -file mapper3.py \
        -file reducer3.py \
        -mapper "python mapper3.py" \
        -reducer "python reducer3.py"
```

- Run this bash file runmr3.sh in the changs/flight directory
  - **bash runmr3.sh**
- **Update your input and output file and directory**
- **Make sure all files: 187.csv are in the HDFS directory changs/flight;**

Lecture 4.3 Page Rank

- What is Page Rank
- How does Page Rank formula work
- Big Data Approach for Page Rank Computation
- Ref. Chap 5 Mining the Massive Datasets
- http://www.mmds.org/#ver21

What is Page Rank

- One of Google founders Larry Page invented this term
- It provides a way to evaluate the importance of each Web page on the Internet
- It is an essential technique for a search engine
- PageRank is a function that assigns a real number to each page in the web

How does Page Rank formula work

- Consider all webpages that form a network
- This network's topology can be represented by a transition matrix *M* (a sparse one)
  - Page *j* provides a link to page *i* represented as *M(i,j)*
  - Assume a random surfer has equal chance to click on any link on a page so $M(i,j)=1/O_j$ *where* O_j *is the number of links on page j (allowing other pages to be visited)*
  - Matrix *M* is *stochastic* since all columns add to 1

Transition matrix *M* example (1)

A Web with 4 pages

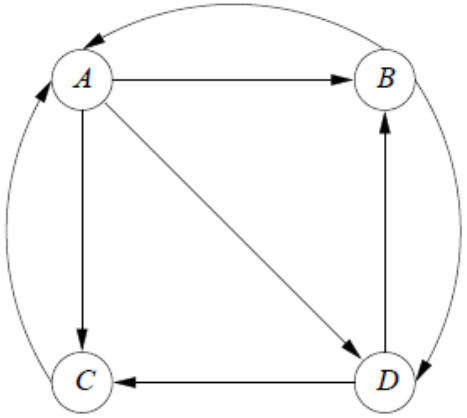


Figure 5.1: A hypothetical example of the Web

The transition matrix

$$M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

Your Turn

A Revised Web

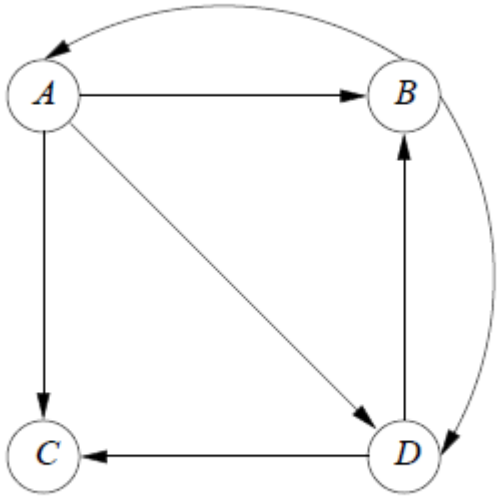


Figure 5.3: C is now a dead end

Write down the M here:

Solving for the Page Ranks

- Let *v* be the vector (n x 1) of PageRank.
- Assume
  - there are *n* pages
  - Network graph is irreducible: (no dead node; reachable to all nodes)
- Let the initial page v_0 be a vector of 1/n
- Solve for *v* by using the Markov concept, i.e. at steady-state, we will have

$$v = M \cdot v$$

Properties of v = M · v

- *M* is *stochastic* (i.e. each column adds up to 1)
- *v* is the principle eigenvector of *M*
- The sum of *v* adds up to 1
- The element values of *v* provide the info where a random surfer is likely to visit in the long run!

PageRank: Three Questions

$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{\mathrm{d_i}}$$

or equivalently

$$r = Mr$$

- Does this converge?
- Does it converge to what we want?
- Are results reasonable?

How do you solve v = M · v

- How do you solve for a small set of linear equations?
- How do you solve for this equation when n is about 1.6 billion (10^9).
  - By iterations
  - Problem: there are infinite solutions if you solves by iterations
  - Ref: http://www.internetlivestats.com/total-number-of-websites/

Your Turn

- Discuss how the mapreduce function can be used to solve $v = M \cdot v$
- Discuss the computation effort in mapreduce when an average web page only contains 10 links.
- Which step (map or reduce) in map reduce can benefit from this fact?
- Assignment: PageRank MapReduce project

Other Issues: Dead ends, spider traps

- Dead end: a page that does not link to others
  - Solution is all zeros
- Spider trap: a page that does not arc out but link back to itself
  - Solution is 1 to node C but 0 everywhere else

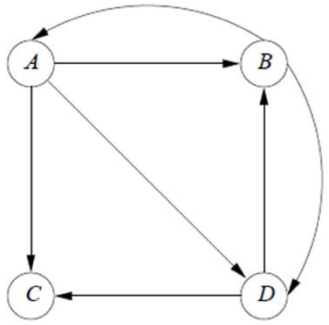


Figure 5.3: C is now a dead end

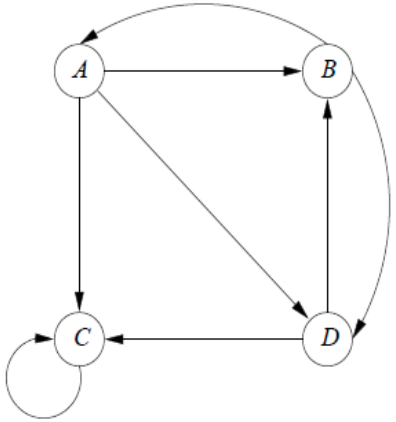
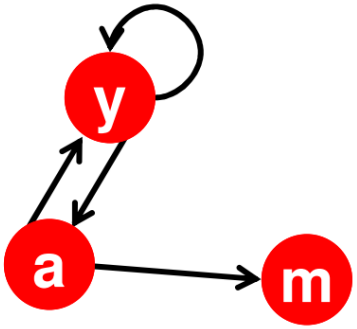


Figure 5.6: A graph with a one-node spider trap

Problem: Dead Ends

- **Power Iteration:**
  - Set $r_j = 1$
  - $r_j = \sum_{i \to j} \frac{r_i}{d_i}$
    - And iterate



	y	a	m
y	½	½	0
a	½	0	0
m	0	½	0

$r_y = r_y/2 + r_a/2$
$r_a = r_y/2$
$r_m = r_a/2$

- **Example:**

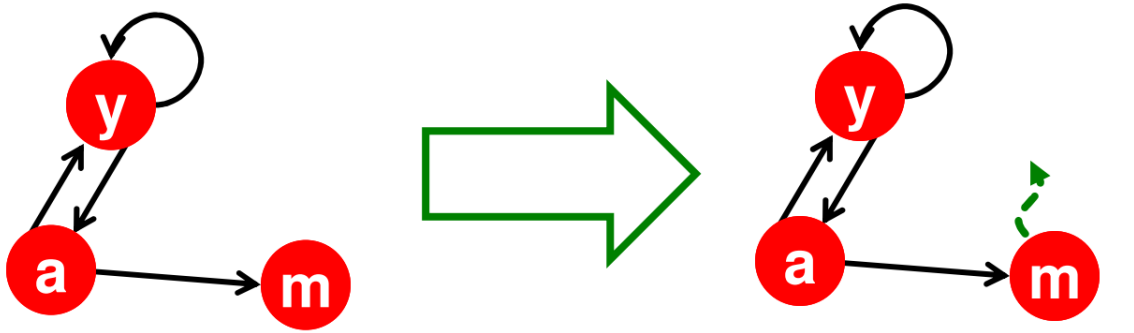
$$\begin{pmatrix} r_y \\ r_a \\ r_m \end{pmatrix} = \begin{matrix} 1/3 & 2/6 & 3/12 & 5/24 & & 0 \\ 1/3 & 1/6 & 2/12 & 3/24 & \dots & 0 \\ 1/3 & 1/6 & 1/12 & 2/24 & & 0 \end{matrix}$$

Iteration 0, 1, 2, …

Here the PageRank "leaks" out since the matrix is not stochastic.

Solution: Always Teleport!

- **Teleports:** Follow random teleport links with probability 1.0 from dead-ends
  - Adjust matrix accordingly



	y	a	m
y	½	½	0
a	½	0	0
m	0	½	0

	y	a	m
y	½	½	⅓
a	½	0	⅓
m	0	½	⅓

Why Teleports Solve the Problem?

Why are dead-ends and spider traps a problem and why do teleports solve the problem?

- **Spider-traps** are not a problem, but with traps PageRank scores are **not** what we want
  - **Solution:** Never get stuck in a spider trap by teleporting out of it in a finite number of steps
- **Dead-ends** are a problem
  - The matrix is not column stochastic so our initial assumptions are not met
  - **Solution:** Make matrix column stochastic by always teleporting when there is nowhere else to go

A revised page rank formula

$$\mathbf{v}' = \beta M \mathbf{v} + (1 - \beta)\mathbf{e}/n$$

Where β is usually between 0.8 and 0.9; e is a vector of 1's

$$\mathbf{v}' = \begin{bmatrix} 0 & 2/5 & 0 & 0 \\ 4/15 & 0 & 0 & 2/5 \\ 4/15 & 0 & 4/5 & 2/5 \\ 4/15 & 2/5 & 0 & 0 \end{bmatrix} \mathbf{v} + \begin{bmatrix} 1/20 \\ 1/20 \\ 1/20 \\ 1/20 \end{bmatrix}$$

Numerical solution

- β=0.8, n=4, then

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} \begin{bmatrix} 9/60 \\ 13/60 \\ 25/60 \\ 13/60 \end{bmatrix} \begin{bmatrix} 41/300 \\ 53/300 \\ 153/300 \\ 53/300 \end{bmatrix} \begin{bmatrix} 543/4500 \\ 707/4500 \\ 2543/4500 \\ 707/4500 \end{bmatrix} \cdots \begin{bmatrix} 15/148 \\ 19/148 \\ 95/148 \\ 19/148 \end{bmatrix}$$

Why Simplified PageRank Doesn't Work

- Spam farm: create one million web pages which all point to one page!
- Spam: techniques for fooling search engines into believing the page is about something it is not!
- How to overcome Spam farm? (Google solutions
  - Pages w/ a large no of surfers are considered important
  - Page content was judged by both terms appear on that page and those used in links to that page

Reading Assignment

- Read chapter 5 of Leskovec and Rajaraman (2nd) http://www.mmds.org/#ver21
- Pay attention to 5.2 on the computational issues of PageRank
- Pay attention to 5.4 the section on Link Spam for a solution about Spam

Computing a large graph

- To compute the PageRank for a large M matrix, the matrix-vector has to be done on the order of 50 times when v is “changed”
- Two issues:
  - M matrix is very sparse
  - MapReduce may not be used for efficiency reason to avoid heavy use of disk (called thrashing)

Representing Transition Matrices

- Why is Web M sparse?
  - An average page has about 10 out-links
- Represent non-zero elements in M

Example 5.7: Let us reprise the example Web graph from Fig. 5.1, whose transition matrix is

$$M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

Recall that the rows and columns represent nodes A, B, C, and D, in that order. In Fig. 5.11 is a compact representation of this matrix.[5]

Source	Degree	Destinations
A	3	B, C, D
B	2	A, D
C	1	A
D	2	B, C

A Map Reduce Strategy

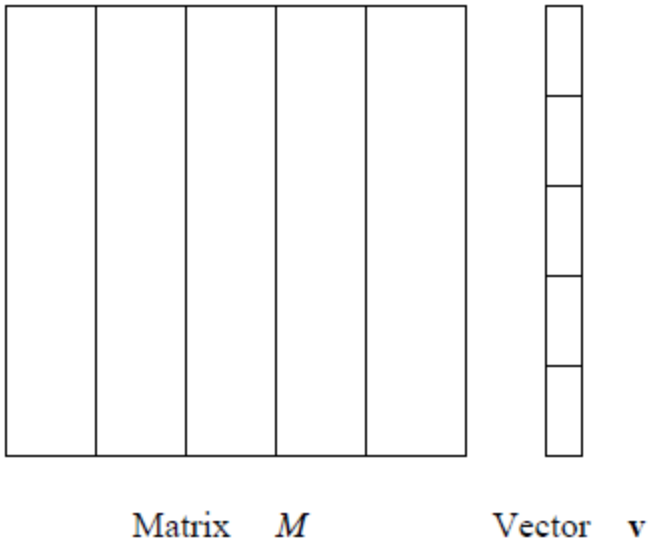


Figure 2.4: Division of a matrix and vector into five stripes

wordcount
is a matrix computation too

$$
\begin{array}{c}
 & \text{term}_1 & \text{term}_2 & & \text{term}_n \\
\text{doc}_1 & A_{1,1} & A_{1,2} & \cdots & A_{1,n} \\
\text{doc}_2 & A_{2,1} & A_{2,2} & \cdots & \vdots \\
 & \vdots & \ddots & \ddots & A_{m-1,n} \\
\text{doc}_m & A_{m,1} & \cdots & A_{m,n-1} & A_{m,n}
\end{array}
= \boldsymbol{A}
$$

$$
\text{word count} = \text{colsum}(\boldsymbol{A}) = \boldsymbol{A}^T \mathbf{e}
$$

e is the vector of all ones

inverted index
is a matrix computation too

$$
\begin{array}{c}
 & \text{term}_1 & \text{term}_2 & & \text{term}_n \\
\text{doc}_1 & & & & \\
\text{doc}_2 & & & & \\
 & \vdots & \ddots & \ddots & \\
\text{doc}_m & & \cdots & & \\
\end{array}
\begin{bmatrix}
A_{1,1} & A_{1,2} & \cdots & A_{1,n} \\
A_{2,1} & A_{2,2} & \cdots & \vdots \\
\vdots & \ddots & \ddots & A_{m-1,n} \\
A_{m,1} & \cdots & A_{m,n-1} & A_{m,n}
\end{bmatrix}
= \boldsymbol{A}
$$

inverted index
is a matrix computation too

$$
\begin{array}{c}
 \\
\text{term}1 \\
\text{term}2 \\
 \\
\text{term}m
\end{array}
\begin{array}{c}
\begin{array}{cccc}
\text{doc}1 & \text{doc}2 & & \text{doc}n
\end{array} \\
\begin{bmatrix}
A_{1,1} & A_{2,1} & \cdots & A_{m,1} \\
A_{1,2} & A_{2,2} & \cdots & \vdots \\
\vdots & \ddots & \ddots & A_{m,n-1} \\
A_{1,n} & \cdots & A_{m-1,n} & A_{m,n}
\end{bmatrix}
\end{array}
= \boldsymbol{A}^T
$$

Matrix-vector product

$$\mathbf{Ax} = \mathbf{y}$$

$$y_i = \sum_k A_{ik} x_k$$

Follow along!

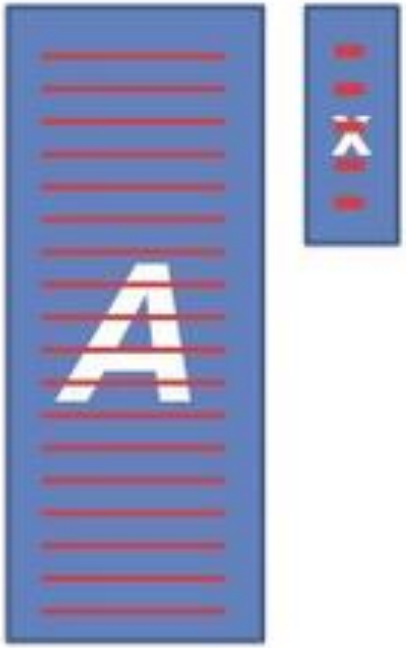
`matrix-hadoop/codes/smatvec.py`

Matrix-vector product

$$\mathbf{Ax} = \mathbf{y}$$

$$y_i = \sum_k A_{ik} x_k$$

Follow along!
`matrix-hadoop/codes/smatvec.py`



A is stored by row

```
$ head samples/smat_5_5.txt
0 0 0.125 3 1.024 4 0.121
1 0 0.597
2 2 1.247
3 4 -1.45
4 2 0.061
```

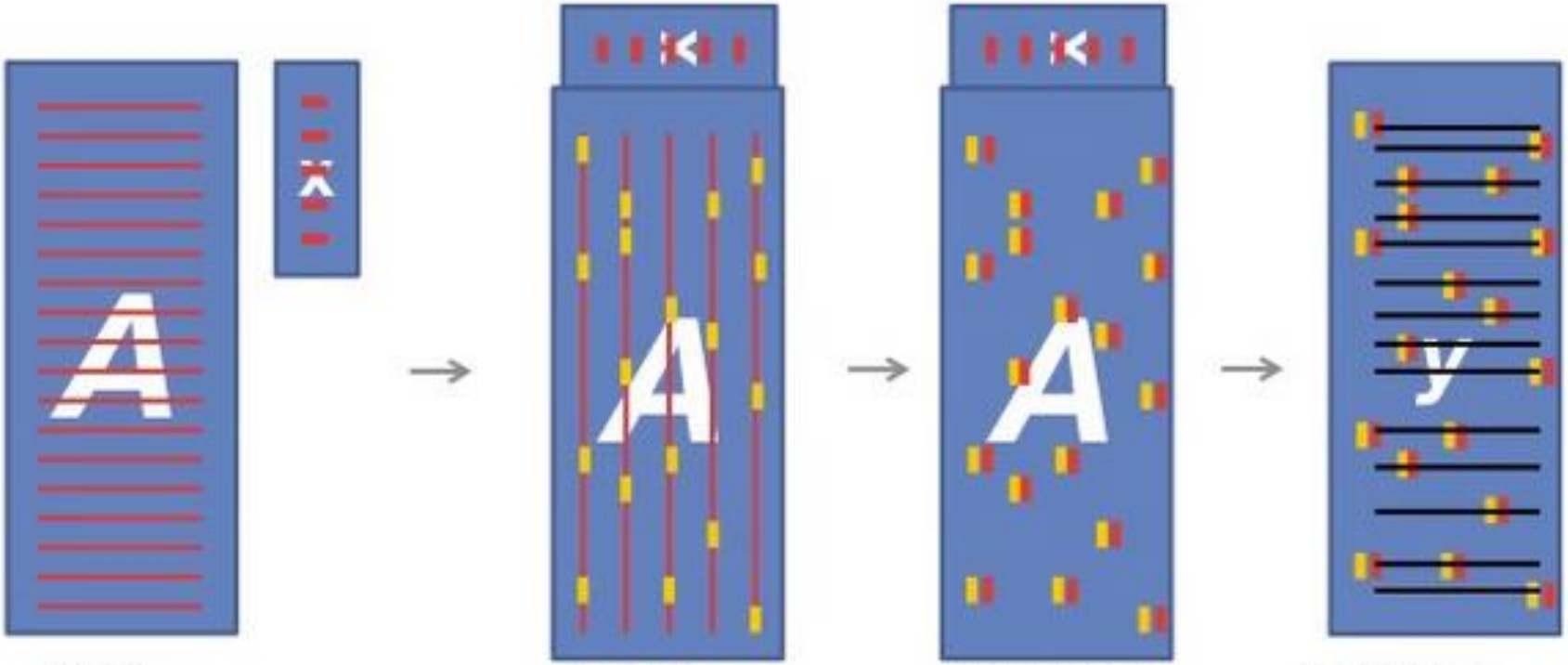
x is stored entry-wise

```
$ head samples/vec_5.txt
0 0.241
1 -0.98
2 0.237
3 -0.32
4 0.080
```

Matrix-vector product (in pictures)

$$\mathbf{Ax} = \mathbf{y}$$

$$y_i = \sum_k A_{ik} x_k$$



Input

Map 1
Align on columns

Reduce 1
Output $A_{ik}\, x_k$
keyed on row i

Reduce 2
Output
sum($A_{ik}\, x_k$)

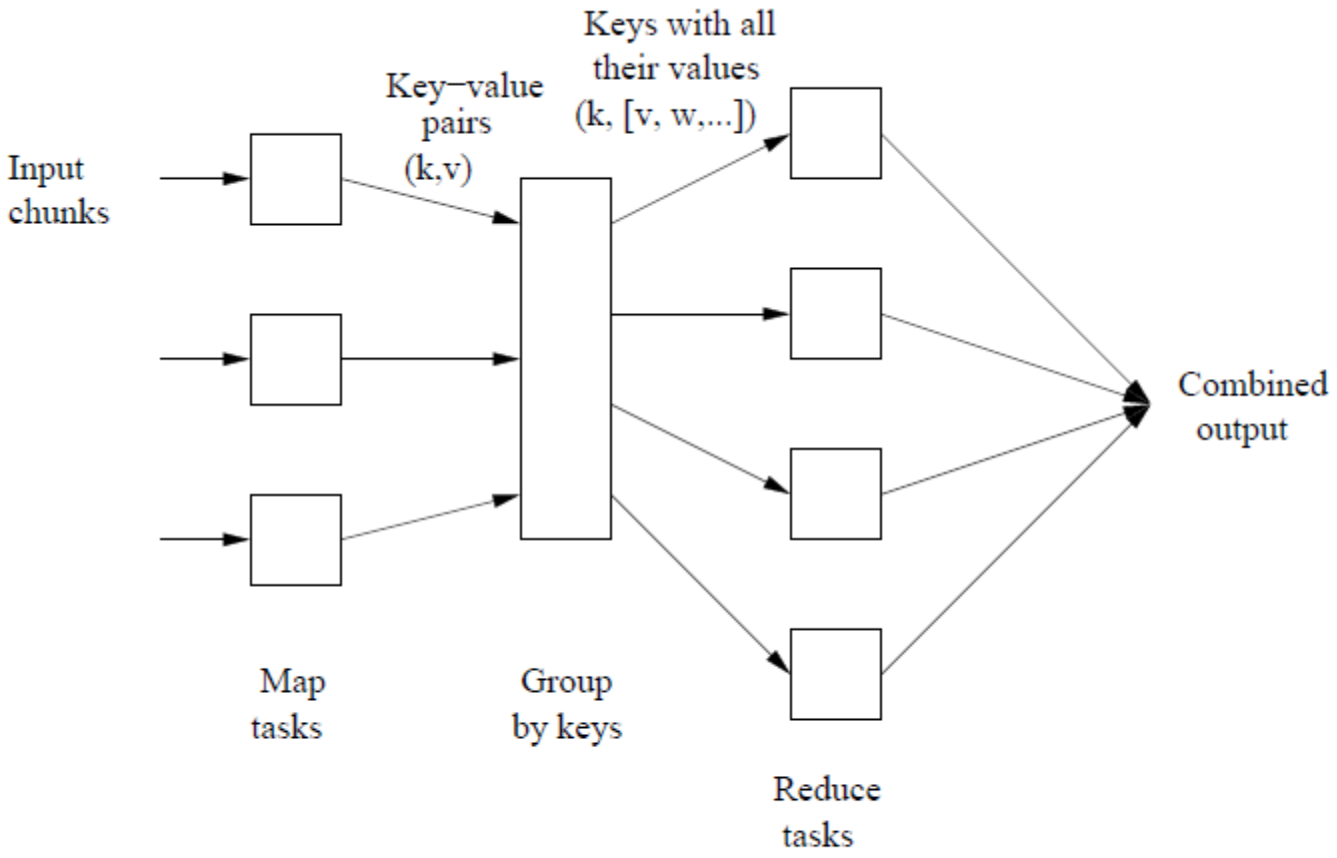


Figure 2.2: Schematic of a MapReduce computation

$$x_i = \sum_{j=1}^{n} m_{ij} v_j$$

The Map Function: The Map function is written to apply to one element of M. However, if $\mathbf{v}$ is not already read into main memory at the compute node executing a Map task, then $\mathbf{v}$ is first read, in its entirety, and subsequently will be available to all applications of the Map function performed at this Map task. Each Map task will operate on a chunk of the matrix M. From each matrix element m_{ij} it produces the key-value pair $(i, m_{ij}v_j)$. Thus, all terms of the sum that make up the component x_i of the matrix-vector product will get the same key, i.

The Reduce Function: The Reduce function simply sums all the values associated with a given key i. The result will be a pair (i, x_i).

Another strategy when a strip in M is still too large

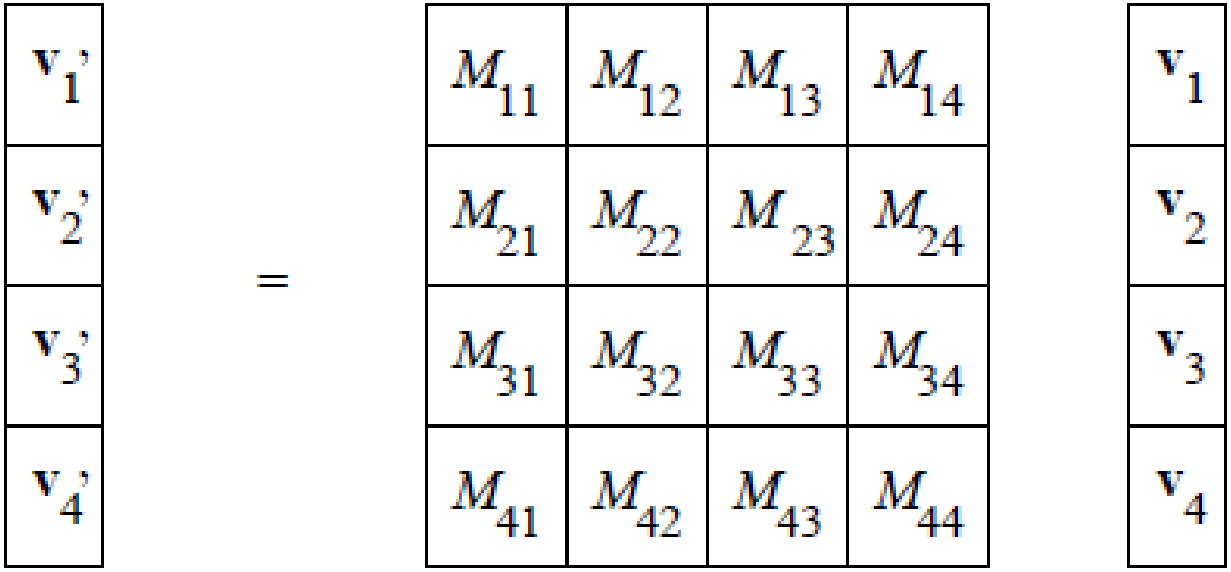


Figure 5.12: Partitioning a matrix into square blocks

The key is to fit both v vectors in the main memory of a node

Why k^2 block partition works?

- Using the k^2 block partition of *M*, we only need to consider a block with at least one non-zero entries.
- This strategy works when k is large.

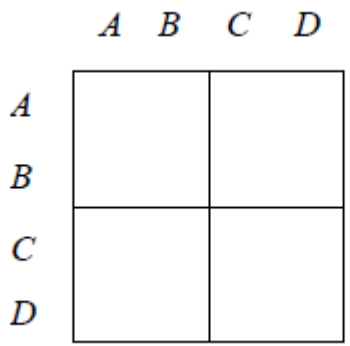
Example 5.8 (k=2)

Example 5.7: Let us reprise the example Web graph from Fig. 5.1, whose transition matrix is

$$M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

Recall that the rows and columns represent nodes A, B, C, and D, in that order. In Fig. 5.11 is a compact representation of this matrix.[5]

Source	Degree	Destinations
A	3	B, C, D
B	2	A, D
C	1	A
D	2	B, C



Your Turn

- Using the k^2 block partition of *M*, describe how your main program needs to be processed in a step-by-step fashion
- Assume that the following equation needs to be repeated at least 50 times for each block

$$\mathbf{v}' = \beta M \mathbf{v} + (1 - \beta)\mathbf{e}/n$$

- How to choose k?